

Short Notes: Computational Thinking & Time Complexity

SCS1304 - Problem Solving Strategies and Computational Approaches

1. Introduction to Computational Thinking

Computational thinking (CT) is a set of skills/processes for solving complex problems systematically, ending in an **algorithm** (a step-by-step solution usable by a person or computer). CT is *not* coding — coding is one way to apply solutions CT produces.

The Four Pillars of CT

Pillar	What it means
Decomposition	Breaking a complex problem into smaller, manageable sub-problems
Pattern Recognition	Spotting similarities/trends across (sub-)problems so solutions can be reused
Abstraction	Ignoring irrelevant detail; keeping only what matters to the solution
Algorithmic Thinking	Designing a precise, ordered sequence of steps to solve the problem

The CT Process (3 stages)

1. **Problem Specification** - analyze and precisely state the problem (uses decomposition, abstraction, pattern recognition).
2. **Algorithmic Expression** - design an algorithm + choose data representations (imperative vs declarative approaches).
3. **Solution Implementation & Evaluation** - implement, test for correctness/efficiency, and generalize if possible.

Imperative vs Declarative Programming

- **Imperative** - *how* to do something (step-by-step instructions, explicit control flow). E.g. C, Java.
- **Declarative** - *what* result is wanted (system decides the “how”). E.g. SQL, Haskell, Prolog.
- Other paradigms: Procedural, Object-Oriented, Functional, Logic, Event-Driven.

Algorithmic Description Tools

- Natural language, programming languages, **pseudocode** (used in this course, similar to C/C++ but with unlimited/variable-sized arrays, free indexing, math notation, flexible control structures).

Key Algorithms Introduced

- **Sequential (Linear) Search** - compare each element with target x until found or array exhausted. Worst case: n comparisons; best case: 1.
- **Binary Search** - works only on a **sorted** array; repeatedly halves the search range (divide-and-conquer). Worst case $\approx \log_2(n)$ comparisons.
- **Fibonacci Sequence** - $f_0=0$, $f_1=1$, $f_n = f_{n-1} + f_{n-2}$. Closely related to the **Golden Ratio** ($\phi \approx 1.618$).

Recursion

A function solves a problem by calling itself on a smaller version of the same problem until a **base case** is reached.

- **Two essential parts:** Base case (stopping condition) + Recursive case (calls itself on a smaller input).
- Managed internally via a **call stack** (LIFO).

Types of Recursion

- **Direct** - function calls itself directly:
 - *Tail recursion* - recursive call is the last statement.
 - *Head recursion* - recursive call is the first statement (work done on the way back).
 - *Tree recursion* - function calls itself more than once (branches).
 - *Nested recursion* - a recursive call's argument is itself a recursive call.
- **Indirect** - function A calls B, which eventually calls A again (a cycle).

Recursion vs Loops: Loops repeat sequentially; recursion naturally handles problems with a *branching* or self-similar structure (e.g., directory trees, Tower of Hanoi, DFS, expression parsing) using the call stack to “remember” return points.

Pros: simpler/more readable code for self-similar problems, shorter code. **Cons:** memory overhead (stack frames), risk of **stack overflow**, generally slower than equivalent loops.

Best practices: clearly define reachable base case(s); ensure every call progresses toward it; watch memory usage (consider memoization); test with varied input sizes.

Example - Fibonacci:

- Recursive: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ (elegant but recomputes values — inefficient, exponential calls).
- Iterative: builds up $f[i]$ from $f[0]$ and $f[1]$ in a loop — efficient, $O(n)$.

2. Time Complexity & Asymptotic Notation

Why Analyze Algorithms?

To judge suitability/efficiency, compare alternatives, and guide improvements. Analysis is based on a **basic operation** (e.g., a comparison or addition) whose execution count is measured as a function of **input size (n)** — independent of CPU/language/OS.

Best / Worst / Average Case

Case	Meaning	Notation	Usage
Best case	Minimum operations (most favorable input)	Ω (Omega)	Rarely used alone; can be misleading
Worst case	Maximum operations (most demanding input)	O (Big-O)	Most commonly used — guarantees an upper bound, vital for reliability/critical systems
Average case	Expected operations over a probability distribution of inputs	Θ (Theta)	Realistic, but hard to compute (needs input distribution)

Example - Linear Search: best = 1 (target is first element); worst = n (target last/absent); average $\approx (n+1)/2$ if present (uniformly distributed).

Asymptotic Notation — Formal Definitions

Let $f(n)$ be a reference function.

- **Big-O ($O(f(n))$)** — *upper bound*: $g(n) \in O(f(n))$ if $\exists c > 0, N \geq 0$ such that $g(n) \leq c \cdot f(n)$ for all $n \geq N$. (g never grows faster than f, eventually.)
- **Omega ($\Omega(f(n))$)** — *lower bound*: $g(n) \in \Omega(f(n))$ if $\exists c > 0, N \geq 0$ such that $g(n) \geq c \cdot f(n)$ for all $n \geq N$. (g never grows slower than f, eventually.)
- **Theta ($\Theta(f(n))$)** — *tight bound*: $g(n) \in \Theta(f(n))$ if $\exists c, d > 0, N \geq 0$ such that $c \cdot f(n) \leq g(n) \leq d \cdot f(n)$ for all $n \geq N$. (g and f grow at the same rate.)
- **Small-o ($o(f(n))$)** — a *stricter* upper bound than Big-O: g grows **strictly slower** than f (ratio $\rightarrow 0$). E.g., $n \in o(n^2)$ but $n \in O(n)$ too (not strict).

Big-O is usually quoted as the **tightest** valid upper bound, even though technically any larger bound also satisfies the definition (e.g. $3n^2+5n+2$ is $O(n^2)$, $O(n^3)$, $O(n^4)$ — but $O(n^2)$ is preferred).

Simplification Rules

1. **Ignore constants:** $5n \rightarrow O(n)$
2. **Ignore lower-order terms:** $n^2 + n + 10 \rightarrow O(n^2)$
3. **Keep only the fastest-growing term.**
4. Logarithm base doesn't matter in Big-O ($\log_2 n$, $\log_{10} n$, $\ln n$ all $\rightarrow O(\log n)$).

Rules for Analyzing Code

- **Sequential statements:** add complexities $\rightarrow O(n) + O(n^2) = O(n^2)$ (dominant term wins)
- **Consecutive loops:** $O(n) + O(n) = O(n)$
- **Nested loops:** multiply complexities $\rightarrow O(n) \times O(n) = O(n^2)$
- Loop variable **increment/decrement by constant** $\rightarrow O(n)$; **multiplied/divided by constant** (e.g. $i*=2$) $\rightarrow O(\log n)$

Common Growth Rates (fastest $\rightarrow$ slowest)

Notation	Name	Typical Example
$O(1)$	Constant	array index access, push/pop
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search, single loop
$O(n \log n)$	Linearithmic	Merge Sort, Quick Sort (avg), Heap Sort
$O(n^2)$	Quadratic	Bubble/Insertion/Selection Sort, nested loops
$O(n^3)$	Cubic	naive matrix multiplication, triple nested loops
$O(2^n)$	Exponential	brute-force subsets/password cracking
$O(n!)$	Factorial	permutations, brute-force TSP

Worked Examples (from notes)

- $n^2 + 10n \in O(n^2)$ — take $c = 11$, $N = 1$ (or $c = 2$, $N = 10$).
- $5n^2 \in \Theta(n^2)$; $n^3 \notin O(n^2)$ (no constant c can bound n^3 by $c \cdot n^2$ as $n \rightarrow \infty$).
- $T(n) = n(n-1)/2$ (e.g., comparisons in a nested loop / selection sort) $\in \Theta(n^2)$.

Practical Notes

- Worst-case (Big-O) analysis is the most important/most used in practice — it gives a performance *guarantee*, crucial for mission-critical/financial/medical systems.
- Average-case is realistic but requires knowing the input distribution; rarely computed exactly.
- Best-case is mostly a theoretical lower bound and can give a misleadingly optimistic impression.

Quick Comparison Summary

Concept	Computational Thinking	Time Complexity
Core idea	A structured <i>process</i> for solving problems	A <i>measurement</i> of how an algorithm's resource use scales with input size
Key tools	Decomposition, Pattern Recognition, Abstraction, Algorithms, Recursion	Big-O, Big-Ω, Big-Θ, simplification rules
Goal	Design a correct, well-structured solution	Evaluate/compare how efficient that solution is as input grows